

Advanced Algorithms

Programming Assignment 1

CJ Kornie 24822874

What I Built

I built a Track B Monte Carlo ray tracing (or technically path tracing) algorithm written in Rust and viewable via a scene loader and basic window output. It is based off the Ray Tracing in One Weekend series (<https://raytracing.github.io/books/RayTracingInOneWeekend.html>) implementing the PPM format (later switching to PNG output using the Rust Image Crate (image-rs Developers, 2026)), the vec3 and ray class, and an initial working demo with sphere and quad primitives. In the current implementation, the program reads a .glb file exported from Blender and renders an output image, either to a file or to the interactive viewport.

I chose Rust and a ray tracer because I have wanted to become more familiar in it and have been looking for an excuse to follow the RTIOW guide.

To preface, I have experience using Blender and a lot of design decisions were made with Blender default behaviour in mind. These include Z being the vertical co-ordinate, handling of the scene object and settings, and a combined material. This later on allowed easy imports of Blender scenes.

Simplifications compared to RTIOW:

- No multiple importance sampling
- Code which was built in the main function and later refactored into its own file has been written directly to its own file in my version (camera.rs, render.rs, sphere.rs, etc.)

Extensions compared to RTIOW:

- Clamping for the contribution of bright rays (preventing fireflies)
- Multithreading
- A SAH-built BVH
- Scene loading from glTF files
- Triangles, extended to full triangle meshes and instancing
- One “Principled” material shader supporting PBR value inputs rather than multiple discrete material shaders written manually in code.
- PNG output

Track Specific Writeup – Track B

Ray Tracing Implementation

render.rs

```
thread::scope(|scope| {
    for _ in 0..threads {
        scope.spawn(|| {
            loop {
                if cancel() {
                    break;
                }
                let Some((y, row)) = rows.lock().unwrap().next() else {
                    break;
                };
                for (x, sum) in row.iter_mut().enumerate() {
                    let mut rng = sample_rng(seed, (y * width + x) as u64, sample);
                    let s = (x as Real + rng.next_real()) * spec.inv_width();
                    let t = (y as Real + rng.next_real()) * spec.inv_height();
                    *sum += shade(s, t, &mut rng);
                    // thread::sleep(Duration::from_micros(1));
                }
            }
        });
    }
});
!cancel()
```

```
pub fn render(
    spec: &ImageSpec,
    settings: &RenderSettings,
    shade: impl Fn(Real, Real, &mut Pcg32) -> Color + Sync,
    mut on_pass: impl FnMut(u32),
) -> Vec<Color> {
    let mut sums = vec![Color::BLACK; spec.pixel_count()];
    for sample in 0..settings.samples_per_pixel {
        render_pass(spec, settings.seed, sample, &mut sums, &shade, &|| false);
        on_pass(sample + 1);
    }
    let samples = settings.samples_per_pixel as Real;
    sums.into_iter().map(|sum| sum / samples).collect()
}
```

The main rendering loop follows the structure of Shirley et al. (2025), but replaces its recursive `ray_color` with an iterative loop that tracks throughput and renders the image progressively (meaning each pass adds one sample to every pixel, rather than finishing all of a pixel's samples before moving on to the next). I use Rust's standard multi-threading functionality to create one worker for each available CPU thread. Each worker takes the next free row of the image from a shared queue and, for every pixel in that row, calls `ray_color()` with a ray from the camera and a reference to the scene. This tests for collision via the `hit()` method in `scene.rs` and `bvh.rs`, which returns a hit record containing the hit point, surface normal, distance and the ID of the material that was hit. At each hit, any emission from the material is added, and a shadow ray is sent towards each light to add its direct light if nothing is blocking it (next event estimation). The material then returns an attenuation and a new scatter direction. The attenuation is multiplied into the throughput and the loop continues with the new ray until it escapes to the sky or reaches the maximum number of bounces. The final colour is added to that pixel's running sum, which is only divided by the number of completed passes when the image is displayed or saved.

scene.rs & bvh.rs

```
pub fn hit(
    &self,
    ray: &Ray,
    t_range: Interval,
    hit_primitive: impl Fn(usize, Interval) -> Option<HitRecord>,
) -> Option<HitRecord> {
    if self.nodes.is_empty() {
        return None;
    }

    let mut range = t_range;
    let mut closest = None;
    let direction = ray.direction();

    let mut stack = [0u32; MAX_DEPTH + 4];
    let mut stack_len = 1;
    while stack_len > 0 {
        stack_len -= 1;
        let node = &self.nodes[stack[stack_len] as usize];
        if !node.bounds.hit(ray, range) {
            continue;
        }

        if node.count > 0 {
            for &i in &self.indices[node.start as usize..(node.start + node.count) as usize] {
                if let Some(hit) = hit_primitive(i as usize, range) {
                    range.max = hit.t;
                    closest = Some(hit);
                }
            }
            continue;
        }

        let (near, far) = if direction[node.axis as usize] < 0.0 {
            (node.start + 1, node.start)
        }
        else {
            (node.start, node.start + 1)
        };
        stack[stack_len] = far;
        stack[stack_len + 1] = near;
        stack_len += 2;
    }
    closest
}
```

Scene immediately calls the `hit()` method in `bvh.rs`, which starts with only the root box of the BVH on a stack. Running down the stack, the while loop performs a slab test on each box to quickly test for intersection, skipping it if the ray misses. If the box has children, both are added to the stack with the closer one on top so it is checked first. If the box is a leaf, it tests for collision with the actual geometry inside it. If the distance is shorter than the closest hit found so far, it records the hit and shrinks the maximum range to that distance, so any boxes further away are skipped from then on. It then continues down the stack until no boxes are left, at which point the recorded hit is guaranteed to be the closest.

principled.rs

The principled material shader takes in PBR values from the object that has been hit and calculates the reflection/scatter or transmission of a ray, returning the combined colour and the direction of the next bounce. The light and emission from each bounce are accumulated back in `ray_color()`. The material randomly chooses whether to perform a metallic, transmissive, specular, or diffuse bounce, weighted by the material parameters. This averages out mixed materials over multiple samples but requires more samples to properly render.

Bounding Volume Hierarchy

O'Kane's (2020) Rust port of the series was consulted for the bounding box and BVH layout, although my version differs. Instead of a pointer based binary tree, my implementation stores the tree in a single flat array, where each node finds its two children by index rather than by pointer. It is also split into two levels: a top-level BVH over every object instance in the scene, and a BVH per mesh over its triangles, with each leaf storing up to 4 objects or triangles. When a ray reaches an instance, it is transformed into that object's own space and continues down the mesh's BVH. This means multiple copies of the same mesh share one BVH rather than duplicating it. Storing every node in one array also keeps them close together in memory, which is generally faster than following pointers.

Interface Decisions

The primary method for a user to interact with the program is via the CLI interface. This uses the `dialoguer` (`console-rs`, 2025a) library to handle user input, selection of different scenes, and setting of parameters. These parameters can also be chosen via CLI arguments and are skipped in the menu if done so. A progress bar is displayed during rendering using `indicatif` (`console-rs`, 2025b).

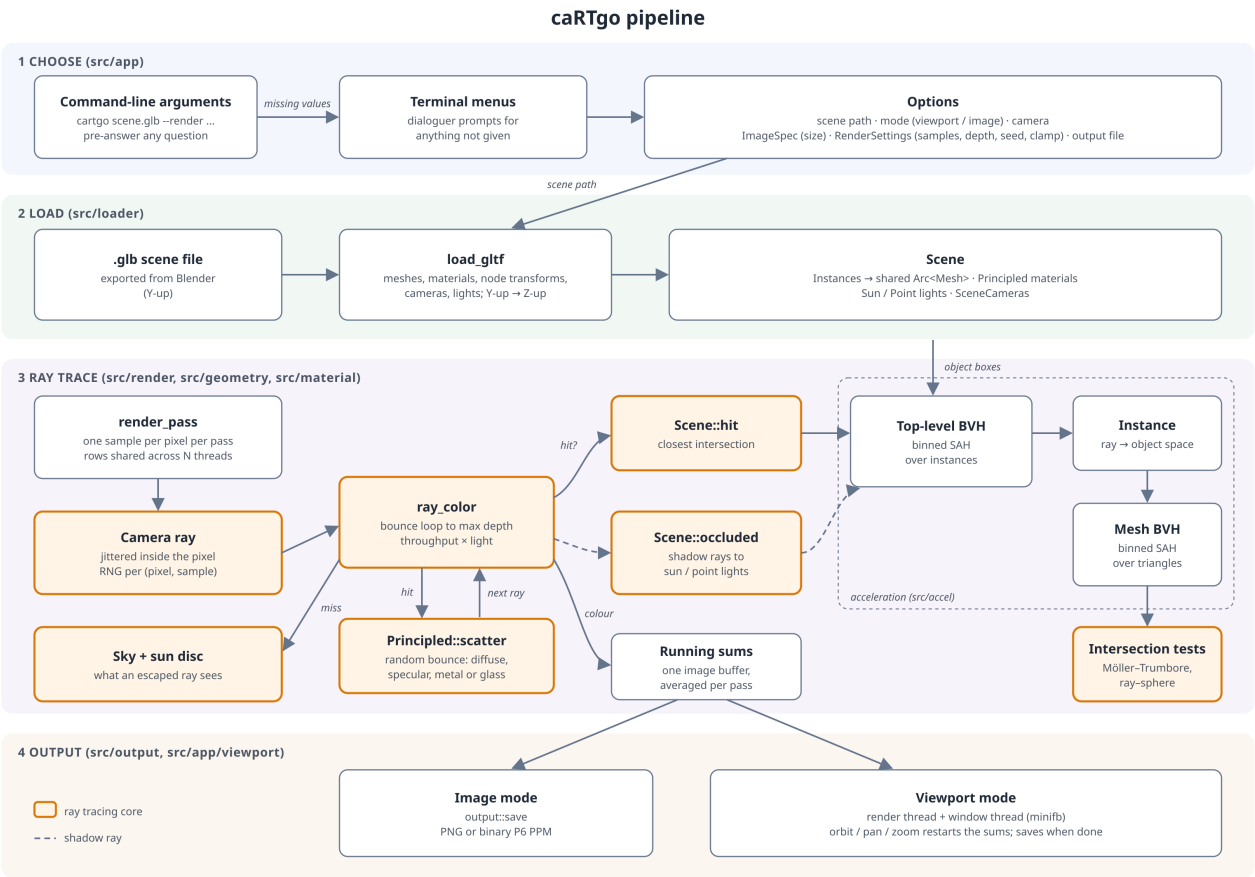
The user is given two options, rendering straight to disk, or to a live viewport with mouse and keyboard controls around a point (mimicking Blender's orbit view). This uses the `minifb` library (Collin, 2025) sending raw RGB values to the frame buffer whenever a rendering pass is complete. On camera movement, the current pass is allowed to complete so intermediary movement frames can be viewed, then the sample count and accumulated buffer are discarded to start rendering again fresh at the new viewpoint.

The choice to progressively render the image stated prior allows this live preview functionality in the viewport. I chose to handle glTF files exported from Blender as this satisfied the requirement for a fully working tool without having to design a method to interact with the scene live. This way, the program acts as a model viewer and rendering engine, rather than 3D editor.

glTF

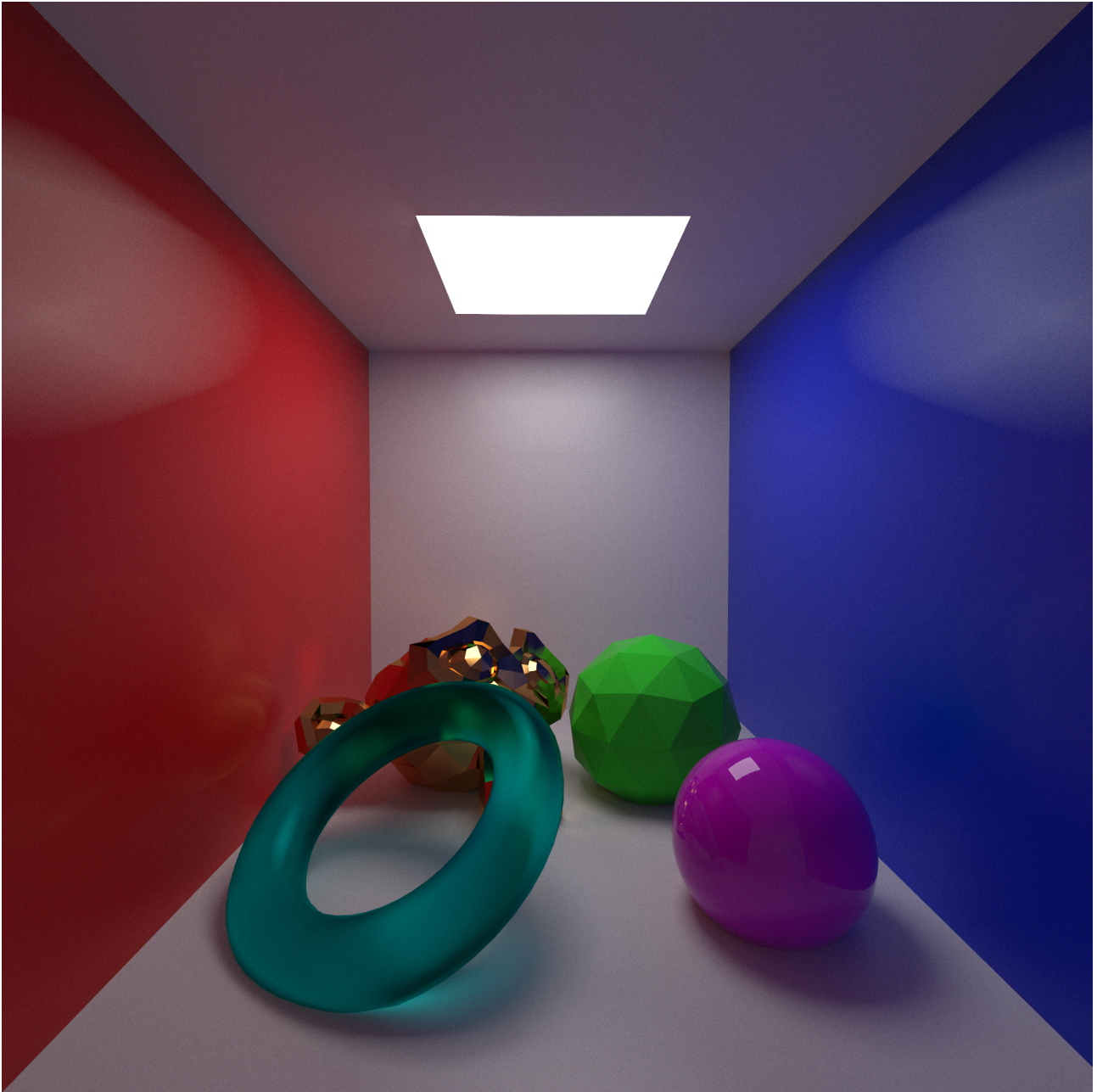
Originally, I was going to only parse `.obj` files as these are very easy to read from and support PBR materials (surprisingly) in a separate `.mtl` file. The issue I ran into was `obj` does not store cameras or light sources meaning these would either have to be hardcoded or I would have to use a different format. I opted to support glTF files using the glTF Crate by Harvey-Macaulay (2024).

Data Pipeline Diagram



Worked Example

Below is a rendered Cornell's Box at 2000 samples and 8 bounces. This is present as a gtb file in the scenes folder and is especially difficult to render due to the lack of optimisations for emissive meshes and enclosed area forcing a full sequence of bounces from each ray. This scene features multiple different material types and meshes.



What I Learned

What surprised me was how simple a Monte Carlo renderer was to implement in code when put into the simple iterative steps as the RTIOW series does. A lot of this can be credited to the quality of the series of course, it wouldn't be the go-to if it wasn't high quality, but I was expecting to have to deal with a lot more complexity to render something good looking.

When I first started implementing the BVH and in the first draft of this document, I wrote it as simply a bounding box around each object to quickly test for rough collision. But as I worked on implementing it myself I realised I was way off basis (and would have to re-write the section in this document). Instead it's a scene wide tree which is split further and further adding to the queue as we iterate pass a ray through and test for intersection.

AI Use

For this assignment I used a Claude Code subscription using the model Opus 5 and 5.5.

What I used it for

For the implementation of RTIOW code (i.e. the ray tracing algorithm), the model was primarily used to plan out code structure with future features in mind. Specifically, the early implementation of BVH acceleration and abstraction of scene, scene parameters, and rendering parameters. Most of the actual implementation of code is already provided via RTIOW.

Outside of the RTIOW code, the model had more of a hand. In the implementation of BVH structure it was especially helpful at fixing a lot of dumb mistakes as I adapted it and the addition of an SAH based split for meshes. Additionally, the importing of OBJ and glTF files was also greatly assisted by the model. In the case of OBJ, handling the import of multiple objects was mostly written by the model after a few failed attempts of my own, and it pointed me towards the glTF Crate by Harvey-Macaulay (2024) and did most of the piping of values.

The TUI was originally made using blood, sweat, and tears but when adopting dialoguer and creating the viewport, I assigned these to the model. I deemed these separate enough from the underlying algorithm to where I could freely assign tasks without worrying about my lack of understanding drawn from working on the code myself.

In this document, all text is hand written and the diagram image used in the Writeup section was created via an AI generated python script which I modified slightly.

Where it was wrong

RTIOW is extremely prevalent within the training data as would be expected, but while this improved the models understanding of the underlying algorithm, this also caused it to be extremely stubborn with certain design decisions I would have rather designed without. A specific example I came across was when designing the piping of scene, camera, and viewport data through the program. In my implementation, these are handled by a `render_setting` and `image_spec` object which is passed through the render sequence. The model itself was very resistant to this, wanting to stick with the textbook, as it often ignored these until I explicitly asked to include them.

When I expressed a want to emulate Blender data structures and functionality, the model took it upon itself to start changing every file in the repo to match as many Blender defaults as it has in its training data. I stopped this before it went too far and reverted all of its changes. This has occurred a few times where the model is seemingly unable to understand nuance.

Additionally, the model needed a lot of guidance and iterative instruction in designing a good TUI. Multiple times it misunderstood instructions which I gave on how I wanted the UI to function. For example, when I wrote “When passing a cli arg, the menu should skip the option.” the model understood it to mean, ‘When a cli arg is passed, skip the menu’. Alongside a few other blunders I had to fix.

What I took on trust

Parameters such as Lumen to Watt conversions for light in the glTF importer and calculation of material properties when importing .mtl files may not be wholly accurate to the exported scene. I did a quick visual check to see if a rendered Blender scene and a scene from my program looked similar but I took the conversion constants at face value.

Additionally, the handling of BVH subdivision via SAH cost is beyond me. I understand the underlying concept but I struggle to understand the actual in-code implementation.

References

- Collin, D. (2025). minifb (Version 0.28.0) [Computer software]. crates.io.
<https://crates.io/crates/minifb/0.28.0>
- console-rs. (2025a). dialoguer (Version 0.12.0) [Computer software]. crates.io.
<https://crates.io/crates/dialoguer/0.12.0>
- console-rs. (2025b). indicatif (Version 0.17.11) [Computer software]. crates.io.
<https://crates.io/crates/indicatif/0.17.11>
- Harvey-Macaulay, D. (2024). gltf (Version 1.4.1) [Computer software]. crates.io.
<https://crates.io/crates/gltf/1.4.1>
- image-rs Developers. (2026). image (Version 0.25.10) [Computer software]. crates.io.
<https://crates.io/crates/image/0.25.10>
- O'Kane, C. (2020). rtiow: Rust port of the Raytracing in One Weekend series, with pictures [Computer software]. GitHub. <https://github.com/okaneco/rtiow>
- Shirley, P., Black, T. D., & Hollasch, S. (2025, April 25). Ray tracing in one weekend (Version 4.0.2). <https://raytracing.github.io/books/RayTracingInOneWeekend.html>